

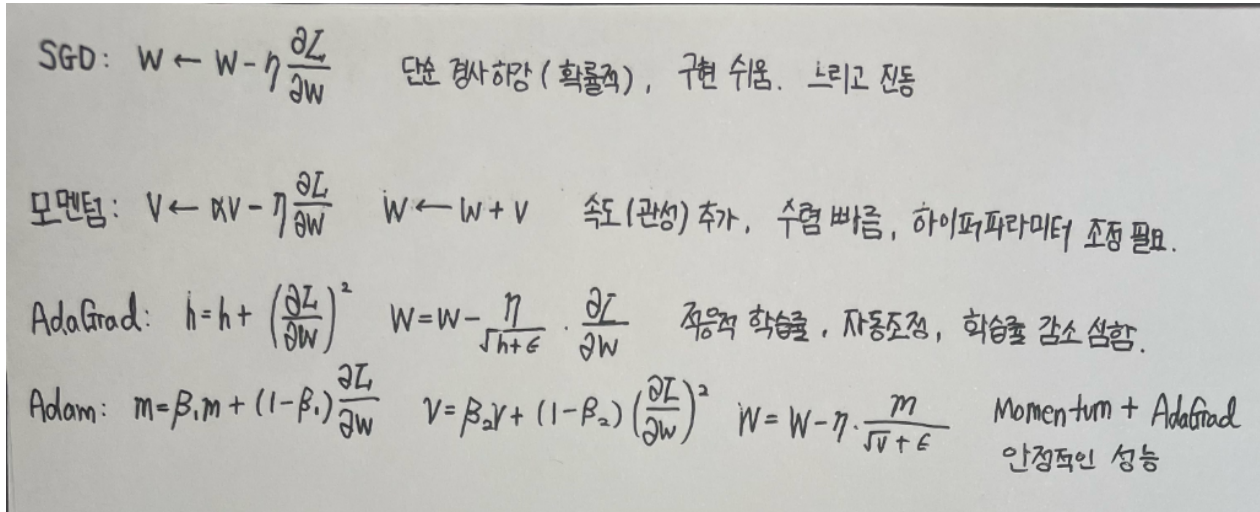
6장. 학습 관련 기술들(1) 실습문제

신경망

학번: 202404252 이름: 원종우

※ hwp 파일 한 개, py 파일 3개 제출하세요.

1. 6.1 절에서 소개한 매개변수 업데이트 방법 4가지의 이름, 수식, 특징을 간단히 적으시오.



※ C 드라이브 아래 Neural_Net 폴더 아래, ch06 폴더를 만들고 아래 문제 2번, 3번 코드를 작성하시오. 교재 Github(<https://github.com/WegraLee/deep-learning-from-scratch/tree/master>)에서 필요한 파일을 다운로드하여 적절한 폴더에 넣으시오.

2. 6.1.7 절의 optimizer_compare_naive.py 코드를 다음과 같이 수정하여 실행하시오.

- 목적 함수를 $f(x, y) = \frac{1}{10}x^2 + y^2$ 로 수정하시오.
- 초깃값(initial value)을 (-5, 1)로 변경하시오.
- 50번 반복하시오.
- optimizer마다 반복 후, optimizer 이름과 그 x_history와 y_history 값을 적절히 출력하시오.

```
[SGD]
x_history: [-5.0, -4.05, -3.2805, -2.6572050000000003, -2.15233685, -1.7433922005000002, -1.412147682405, -1.14383962274885, -0.9265100944259206, -0.7584731764849957, -0.6078832729528465, -0.49238545109180565, -0.3988322153843626, -0.3230540944613337, -0.2616738165136803, -0.21195579137680106, -0.17168419181462565, -0.13966419472184678, -0.1126419977246959, -0.09124001815700367, -0.07390441478717297, -0.059862575912810106, -0.04848868648937618, -0.0392758360539471, -0.031813427285679715, -0.02576887603660057, -0.02087278958964646, -0.01690659567613632, -0.013694637249767043, -0.01092656172311385, -0.008985051499572157, -0.007277891714653447, -0.005895092288869292, -0.004775024753984127, -0.00386770050727143, -0.0031328937410889856, -0.0025376439302820784, -0.0020554915835284833, -0.0016649481826580715, -0.001348680027953828, -0.0010923725026419607, -0.0008848217271399882, -0.0007167855989833904, -0.0005805315351765462, -0.000470238543438024, -0.0003808067402203117, -0.000308512259585759, -0.00024097902644647, -0.00020241830112164, -0.00016395925232925133]
y_history: [1.0, -0.8999999999999999, 0.8089999999999998, -0.7289999999999999, 0.6560999999999999, -0.59049, 0.5314409999999999, -0.4782689999999999, 0.4304672099999999, -0.3874268999999999, 0.3486704409999999, -0.3138185689999999, 0.2824295364089999, -0.2541865823899974, 0.22876792454960976, -0.20589113209464877, 0.18530281888518388, -0.1667718169966655, 0.1500946352969989, -0.13508517167299, 0.121576654905691, -0.10941898913151218, 0.0984778902136096, -0.08862938119652486, 0.07976644307687236, -0.07178979876918512, 0.0646108188926661, -0.058149737003039936, 0.05233476330273594, -0.04710128697246234, 0.0423911582752161, -0.03815204244769449, 0.03433683820292503, -0.030903154382632525, 0.02781283894436927, -0.02503155504993234, 0.02252399544939143, -0.02027555959044519, 0.018248003631400674, -0.01642320326826066, 0.014780882941434544, -0.013302794647291089, 0.011972515182561979, -0.01077526366430578, 0.0096977372975282, -0.00872796356808768, 0.00785516721127891, -0.007069650490151019, 0.006362685441135916, -0.005726416897022324]

[Momentum]
x_history: [-5.0, -4.9, -4.712000000000001, -4.4485600000000005, -4.122492000000001, -3.7465824640000007, -3.333315123200005, -2.8947390255616003, -2.4421110069678003, -1.985903570094039, -1.535598805505766, -1.099612541266205, -0.685232652625276, -0.29858609979593437, 0.05336751974639176, 0.37281842693955747, 0.6510678748746155, 0.8884710205186753, 1.0843644311879557, 1.2389812121665489, 1.353366908039518, 1.4292274877615354, 1.46892665526813, 1.4752773729187023, 1.451487413458434, 1.4010468105033536, 1.3276292795350457, 1.2350009160728677, 1.1269353706354501, 1.0071376723290653, 0.8791769984067376, 0.7464288368685079, 0.6120269219467311, 0.47882466007819735, 0.34936613119495297, 0.22586613257613397, 0.1101988111676422, 0.0038942456706945, -0.09185774817869774, -0.176197387684988, -0.2485791154869495, -0.35673084187582005, -0.39277800334746343, -0.4173648868049932, -0.43114578756467015, -0.43492568007886, -0.42962987124871853, -0.41626954114317344, -0.3959285732325194]
y_history: [1.0, 0.0, 0.4599999999999999, 0.061999999999999983, -0.208680000000000026, -0.5804200000000004, -0.7089740000000003, -0.5828778000000002, -0.52281556, -0.2741966019999999, 0.00439987604000189, -0.25425672182000025, 0.4282765435400024, 0.49923907440380016, 0.46325753728786007, 0.33822264642594196, 0.1580467153650873, -0.03572096566280136, -0.2029670854552869, -0.3128961961774665, -0.34925261659193485, -0.3112228716456594, -0.2162815268664266, -0.08676801119101277, 0.047147755155062254, 0.15824239383551733, 0.2265790890882343, 0.2427626934543423, 0.2087815262944971, 0.13643892618975426, 0.044842800857534856, -0.04792227211296958, -0.12116838336382966, -0.1627580868168378, -0.16768062656117755, -0.13858133901884784, -0.08467571242698152, -0.01922550608095532, 0.04352478096914396, 0.09129508385555971, 0.1160293383222195, 0.11508430039777358, 0.09121690618621532, 0.05149287015856982, 0.005442663701974902, -0.0370910548493555, -0.06795319057568176, -0.08213847461423904, -0.07847753532609278, -0.05948718790154259]

[AdaGrad]
x_history: [-5.0, -3.5000001499999985, -2.6398066791828856, -2.0443352188588288, -1.6035468369121273, -1.2666330683359703, -1.0045987601370454, -0.7987391933453749, -0.6360295781609269, -0.5069458663486073, -0.4043011713511223, -0.3225612641525765, -0.2574087324790812, -0.2054472388007307, -0.1639907295936605, -0.13090763973876107, -0.10450274380820174, -0.0834259627581293, -0.06660114004546261, -0.053169980067681384, -0.04244769719594582, -0.03388781424774691, -0.0270541640080523292, -0.021598590323170276, -0.01724317856541825, -0.01376064551521721, -0.01099009466883704, -0.008773922328943994, -0.007004646354761439, -0.005592148438347455, -0.00446448325936621, -0.0035642136290998825, -0.0028454847793533853, -0.00227168869680077, -0.0018135994220486143, -0.0014478845218608716, -0.0011559165564284492, -0.000922824347476586, -0.0007367355140949561, -0.0005881717574161865, -0.00046956609241266474, -0.000374877427348744, -0.00029928756780754, -0.00023983205334540342, -0.0001907510677988213, -0.000152285849198524, -0.00012157719033544266, -0.706097844437941e-05, -7.4884952579502e-05, -0.18628309058647e-05]
y_history: [1.0, -0.49999999250000038, 0.178028377514904, -0.05573019085512623, 0.018092424299452153, -0.00587053183345144, 0.001904728246880266, -0.0006179975252369045, 0.000205119153954167, -0.505693631688212e-05, 2.118799712860801e-05, -0.8485786096439345e-06, 2.222058178936913e-06, -7.20953546033016e-07, 2.3391641582149044e-07, -7.589516674364503e-08, 2.4624506641898725e-08, -7.989524937800953e-09, 2.592235030736898e-09, -8.41615783657674e-10, 7.28859729983768e-10, -8.853900377183345e-11, 2.8726852841774204e-11, -3.20548447999127e-12, 3.0240912170222067e-12, -0.81719137675538e-13, 1.834770551586937e-13, -1.0328925444471904e-13, 3.3512633824257586e-14, -1.0073315253135656e-14, 3.527892949687956e-15, -1.1446397326582432e-15, 3.7138318431567534e-16, -1.2049683901164343e-16, 3.909570714288552e-17, -1.2684766916205788e-17, 4.1156260745049445e-18, -1.33533221477785e-18, 4.3325416581516836e-19, -1.4057113965876114e-19, 4.560889857292889e-20, -1.4797999319670928e-20, 4.801273232126614e-21, -1.557793308114364e-21, 5.054326181007438e-22, -1.63989740038355e-22, 5.320716214271764e-23, -1.7263287950937117e-23, 5.601146516282625e-24, -1.81731558826454e-24]

[Adam]
x_history: [-5.0, -4.700000948600298, -4.400628486387124, -4.102361365730004, -3.805727692925009, -3.51130878787603045, -3.2197370903236067, -2.931770058325405, -2.6479665759212194, -2.369321433395951, -2.096631545134877, -1.8308066715774787, -1.5727992038686311, -1.323594001051746, -1.084194639734347, -0.855608268232947, -0.638822436172382, -0.43478574275878856, -0.2443817910003982, -0.06840441895384544, 0.09246668053400578, 0.237688093781807, 0.36687160693636, 0.4797955758966712, 0.576417321295182, 0.6568706159747467, 0.7214640713358265, 0.770671677089378, 0.805119560193917, 0.8255696256859616, 0.8329812083647378, 0.8280917644175377, 0.8121974526183581, 0.7863342382761171, 0.751659936253155, 0.7093574184090269, 0.6606198608878246, 0.6066324032377777, 0.548560932764094, 0.4875618878272314, 0.42471496357843035, 0.36107199607065193, 0.2976171823144201, 0.23526456157633302, 0.17485012551847308, 0.117125078346087272, 0.0627502834698343, 0.012291656113486094, -0.03378222630933974, -0.075102137670436403]
y_history: [1.0, 0.7000004743408091, 0.40728468197266415, 0.1319627396825228, -0.11019192077766946, -0.3002127338680726, -0.42439276104936585, -0.4814894451325069, -0.48052554633605765, -0.434279375300238, -0.3553289600483318, -0.2550474418543535, -0.14383910542386058, -0.0315504180832154, 0.07267662137359517, 0.1605420570807077, 0.22577363776294956, 0.2647139007715013, 0.27660756134197945, 0.2632728729276107, 0.22845806109477072, 0.1771908460656206, 0.11526142004724482, 0.048810999589287356, -0.016036871830722874, -0.07365806407100873, -0.11937238328616698, -0.14990538019216057, -0.16366266774588545, -0.1607756163193894, -0.14293219293985363, -0.11306921487992713, -0.0750020983530601, -0.0338310644255572, 0.008451476605097465, 0.04538887251953256, 0.0744260848796018, 0.0932360153896453, 0.10071077873223905, 0.09708624500952957, 0.08341701274013861, 0.06219372926411749, 0.036209337852527675, 0.008640830923628572, -0.01738504333087076, -0.039125980434661835, -0.054507142087214365, -0.062317394547034125, -0.0622939646997563, -0.055809808680214022]
```

3. 6.1.8 절의 optimizer_compare_mnist.py 코드를, 100개 노드를 가지는 은닉층을 3개를 두도록 수정하여 Visual Studio에서 실행하시오. 어느 optimizer가 좋은지 판단하시오. 왜 좋은가?
아담이 가장 좋다. Adam은 모멘텀과 적응적 학습률 조정(AdaGrad의 장점)을 결합한 방식으로, 손실이 가장 빠르고 안정적으로 줄어들며, 오버슈팅이나 진동이 적다.
따라서 아담이 MNIST 분류 문제에서 손실률이 가장 낮고 수렴 속도가 가장 빠른 optimizer이다.

4. (팀플 관련) Neural_Net 폴더 아래 data 폴더를 만들고 작성하시오.

- (1) <https://github.com/zalandoresearch/fashion-mnist/tree/master> 에서 data/fashion/ 폴더에서 4개의 압축 파일을 다운받으시오.
- (2) Neural_Net/data/fashion/ 에 (1)에서 다운로드한 4개 파일을 넣으시오.
- (3) 위 사이트에서 mnist_reader.py 파일을 다운로드하고 data 폴더 안에 넣으시오
- (4) optimizer_compare_mnist.py 파일을 fashion mnist 데이터에 대해 실행하는 것으로 코드를 수정하여 optimizer_compare_fashion_mnist.py에 저장하여 제출하시오.

```
import sys, os
```

```
import matplotlib.pyplot as plt
```

```
sys.path.append(os.path.join(os.path.dirname(__file__), '..'))
```

```

from data.fashion import mnist_reader
from util import smooth_curve
from multi_layer_net import MultiLayerNet
from optimizer import *

x_train,      t_train      =      mnist_reader.load_mnist('../data/fashion/',
kind='train')
x_test, t_test = mnist_reader.load_mnist('../data/fashion/', kind='t10k')

train_size = x_train.shape[0]
batch_size = 128
max_iterations = 2000

# 1. 실험용 설정=====
optimizers = {}
optimizers['SGD'] = SGD()
optimizers['Momentum'] = Momentum()
optimizers['AdaGrad'] = AdaGrad()
optimizers['Adam'] = Adam()
# optimizers['RMSprop'] = RMSprop()

networks = {}
train_loss = {}
for key in optimizers.keys():
    networks[key] = MultiLayerNet(
        input_size=784, hidden_size_list=[100, 100, 100, 100],
        output_size=10)
    train_loss[key] = []

# 2. 훈련 시작=====
for i in range(max_iterations):
    batch_mask = np.random.choice(train_size, batch_size)
    x_batch = x_train[batch_mask]
    t_batch = t_train[batch_mask]

    for key in optimizers.keys():
        grads = networks[key].gradient(x_batch, t_batch)
        optimizers[key].update(networks[key].params, grads)

    loss = networks[key].loss(x_batch, t_batch)
    train_loss[key].append(loss)

    if i % 100 == 0:
        print("======" + "iteration:" + str(i) + "=====")

```

```
for key in optimizers.keys():
    loss = networks[key].loss(x_batch, t_batch)
    print(key + ":" + str(loss))
```

3. 그래프 그리기=====

```
markers = {"SGD": "o", "Momentum": "x", "AdaGrad": "s", "Adam": "D"}
x = np.arange(max_iterations)
for key in optimizers.keys():
    plt.plot(x, smooth_curve(train_loss[key]), marker=markers[key],
markevery=100, label=key)
plt.xlabel("iterations")
plt.ylabel("loss")
plt.ylim(0, 1)
plt.legend()
plt.show()
```